Name: _____

Collaborators: _____

Thayer School of Engineering • DARTMOUTH

*ENGS 28, W26*

# Lab 6 — DC Motors

Due: Tuesday February 17, 2026 2:15 PM

## Contents



## Deliverables

# 1 Overview

## 1.1 Place in course

You have now covered timers, interrupts, and more sophisticated communications (Two-Wire/I2C). You have written and examined a few device drivers. This lab will help solidify your ability to write device drivers and will allow you to assemble many of the sub-components you have studied into a sophisticated motor control system with visual feedback to the user.

## 1.2 Learning Objectives

- Control the speed of a DC permanent magnet motor with a PWM signal.
- Measure the speed of a DC permanent magnet motor with an optical sensor.
- Increase proficiency with device drivers, timers, and interrupts.

# 2 Design Challenge

Develop a prototype motor control and display system that includes the following:

- A modular tb6612 device driver that includes:
  - An initialization function.
  - A mode select function.
  - A speed select function. This should configure Timer 14, Channel 1 as a 1.6KHz PWM generator.
- A potentiometer that sets the speed of the motor:
  - When the potentiometer is set to 0V, the motor should go full speed counterclockwise.
  - When the potentiometer is set to 1.65V, the motor should be shut off (this should include a dead zone).
  - When the potentiometer is set to 3.3V, the motor should go full speed clockwise.
  - Generally, motor direction matches the direction the potentiometer knob is turned.
- A measure of the speed of the motor in RPM, displayed on the seven segment display.

Build and test incrementally, following the steps below. Double-check your wiring before you apply power—it is very easy to damage components when working with high current. The schematic you are building is shown in Figure 1. An example breadboard layout can be seen in Figure 4 (Appendix A).

**Pro Tip!**

Do not leave your motor running for a long period of time, as the voltage regulator can get very hot. If you notice a "hot smell" or components that are hot to the touch, disconnect the power and recheck your wiring!

*Proceed systematically through the design, with incremental design-test-debug.* This is an important discipline to master. It is faster in the long run to take steady steps forward than to move too fast and have to backtrack.

## 2.1 Motor Controller Device Driver

Start with a blank breadboard.

Here are prototypes for the motor controller driver functions. The mode and speed (`PWM`) controls are separated for flexibility. Put these, plus any necessary #define and #include preprocessor directives, in a header file, `tb6612.h`.

```
void motor_init(void);                  // Set ports for PWM, IN1, IN2
void motor_mode(uint8_t mode);          // FWD, REV, BRAKE, STOP

void motor_speed(uint16_t pwm_value); // interacts with TIM14
```

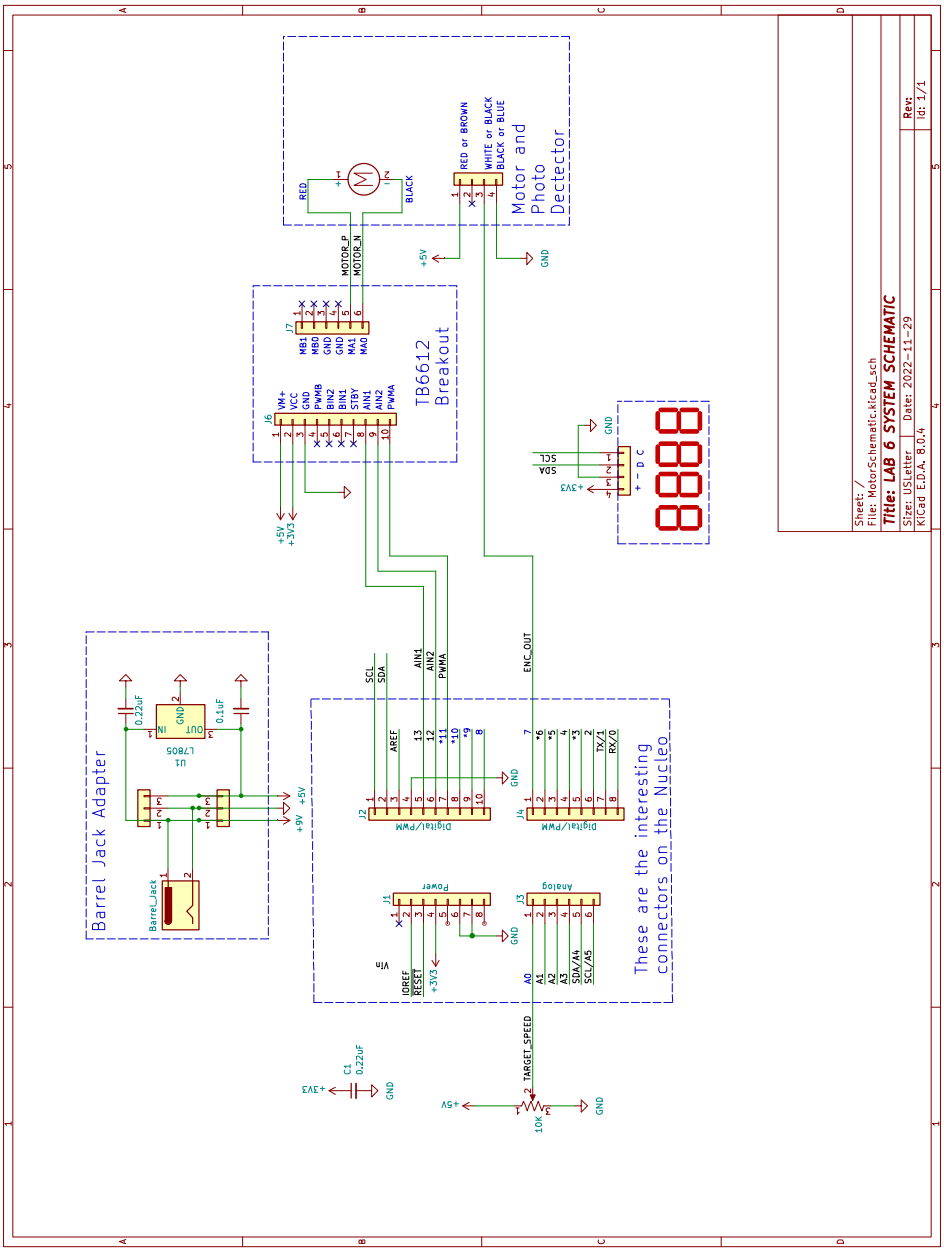


Figure 1: Motor Controller Schematic

Use TIM14 CH1 for the PWM. Design for a $1.6KHz$ PWM frequency, with 1250 discrete PWM values.

Test this functionality without connecting the motor. You should be able to observe the output pins from your Nucleo using Waveforms. Testing behavior before connecting your board to a high current system is good practice.

Put the function implementations in a file `tb6612.c`. Test your drivers by writing a program to exercise the different modes and a few speeds and observe the outputs with Waveforms to confirm proper function. After testing, add them to your library in `Library`.

**Deliverable 1:** Motor Driver *(8 points)*

> Submit screenshots showing a few different PWM settings. Be sure to indicate the PWM frequency and duty cycle in your screenshots. Also note the settings passed to the driver for each of the screenshots.
> Submit your `tb6612.c` and `tb6612.h` files.

## 2.2 Potentiometer Control

The basic idea here is that when the potentiometer is turned all the way to the left (counter clockwise), the motor will turn at maximum speed in the counter clockwise direction (or, "reverse"), and when the potentiometer is turned all the way to the right (clockwise), the motor will turn at maximum speed in the clockwise direction (or, "forward"). So that the motor doesn't dither between forward and reverse when the pot is centered, design in a small dead band where the motor is stopped, between the forward and reverse zones. Clockwise and counter-clockwise are when viewing the motor as seen in Figure 2.

Figure 2: Wheel position

In your main loop, use the value returned by `milliseconds()` to sample your ADC at a 100Hz rate. Later, you will add other functionality that runs at a different frequency, so you may *not* use delay_ms() for timing.

Map the ADC output to the appropriate settings for the `IN1`, `IN2`, and `PWM` signals.

Test your code using Waveforms, again without connecting the motor driver or motor.

## 2.3 Voltage Regulator

Once you have the ADC and potentiometer working, connect the voltage regulator.

In your kit, you should find a small circuit board with a barrel jack and regulator on it. (Figure 3). Notice that it is sized to fit across the gap in the center of your breadboard. Place it in the breadboard at one end. Use the $5V$ pin to power the motor only. Connect the ground to the ground of your breadboard.

> **Important!**
>
> DO NOT connect the $5V$ output to your power rail on the breadboard. You will damage the Nucleo.
> ONLY connect it to the Motor Power on the TB6612

Double check that you have it wired correctly. Once you are sure (ask an instructor if you are not sure) power your regulator using the 9V "wall wart". You should be able to measure 9V on the input side of the regulator and 5V on the output side. Note: Even though the Nucleo provides 5V, it is not able to supply enough current to drive the motor.

Continue to power your Nucleo with the USB cable.

Figure 3: Adapter Board

### 2.4 Motor Driver

When you have this working, wire in the motor driver breakout and the motor.

Now you can test your $POT \rightarrow ADC \rightarrow PWM \rightarrow BREAKOUT \rightarrow MOTOR$ system and confirm that it works.

**Important!**

If you notice a "hot smell" or a component is hot to the touch, disconnect the power and recheck your wiring!

### 2.5 RPM indicator

Wire the speed sensor to power and ground, and pull the `OUT` terminal up with an internal pullup resistor or a $10k\Omega$ resistor to $3.3V$. Confirm with Waveforms that you get a square wave signal with frequency proportional to motor speed.

In the reading quiz, you derived an expression to convert the number of sensor output pulses per second (PPS) to revolutions per minute (RPM):

$$RPM = 60 \times \frac{PPS}{20}$$

Write code to detect pulses, count them for one second, convert to RPM, and display on your computer screen. Use negative RPM to indicate the motor is running in reverse. Use the `milliseconds()` function again to time your RPM calculation.

**Note!**

The photo interrupter requires a pullup on the output to function correctly. You can either use the internal pullup, or an external one (connected to $3.3V$).

Confirm that your design works. At this point, you're done with the main part of the lab and can demonstrate to an instructor.

**Deliverable 2:** Final Design *(8 points)*

Submit your final code including all features, along with any supporting screenshots.

**Deliverable 3:** Demonstration *(4 points)*

Demonstrate your working design to an instructor or TA.

Signature _____

## 3 Final Touches

Display the RPM, positive and negative, on your seven-segment display. Plan ahead for this as you wire up your breadboard.

**Deliverable 4:** RPM Display *(2 points)*

Demonstrate your RPM display to an instructor.

Signature _____

# 4 Going Further (optional)

We are running this motor using very high frequency (for a motor) PWM. This provides very smoothly angular velocity and torque, at the expense of low speed operation. Some sources [1] recommend running at a lower PWM frequency for improved low speed torque.

If you are curious, change your PWM to run at $50Hz$ and compare the low speed (and high speed) behavior of your motor.

[1] See https://learn.adafruit.com/improve-brushed-dc-motor-performance/pwm-frequency

# A Breadboard Layout

Here is an example breadboard layout to help you get started.

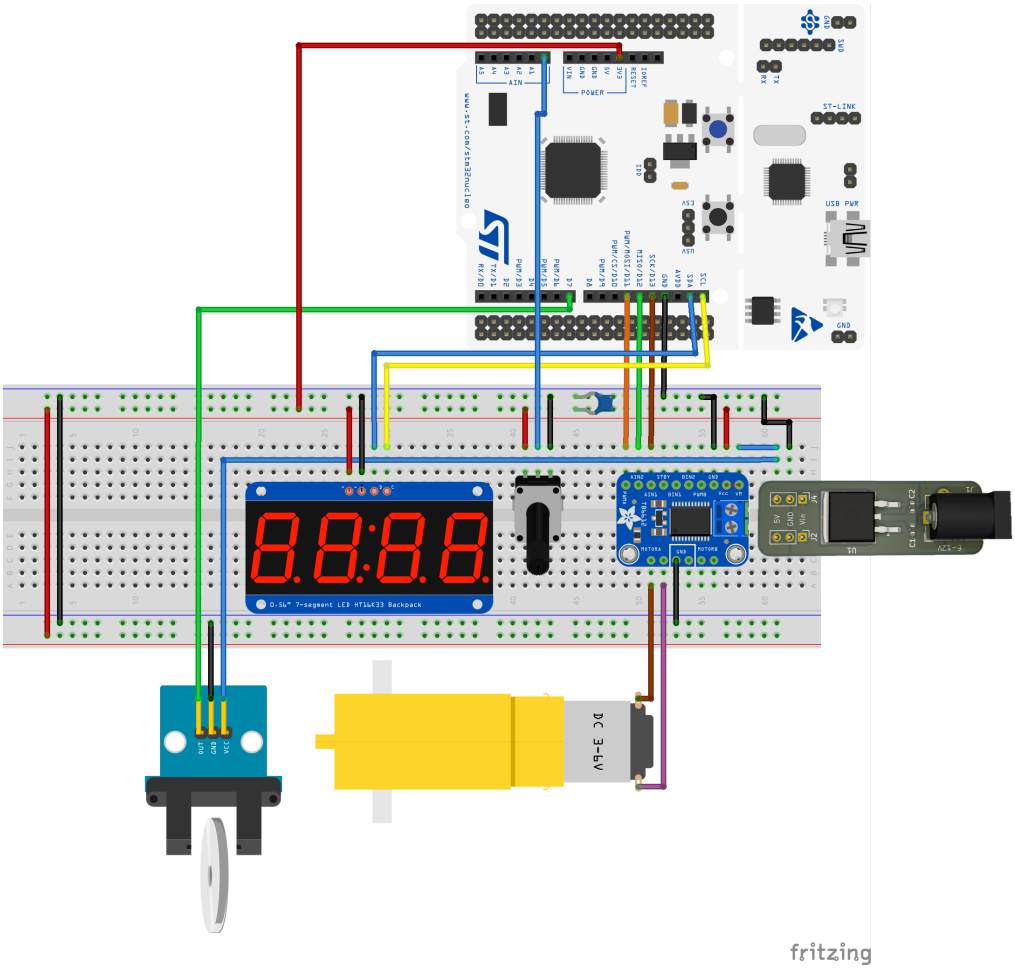


Figure 4: Breadboard Layout for Motor Controller